使用 Claude Code 实现长时运行 Agent Harness：Subagent + Skills 完整实践指南

概述

Anthropic 提出的「Effective Harnesses for Long-Running Agents」模式解决了 AI Agent 跨多个 context window 持续工作时面临的核心挑战：**每个新 session 启动时都没有前一 session 的记忆**。该方案采用双 Agent 架构——**Initializer Agent** 负责首次环境搭建，**Coding Agent** 在后续每个 session 中增量推进并留下结构化的进度记录。

本文详细介绍如何使用 **Claude Code** 原生的 **Subagent** 和 **Skills** 功能将此模式落地为可复用的工程实践。

核心问题与架构思路

长时运行 Agent 的失败模式

根据 Anthropic 的研究，直接让 Claude 在循环中跨多个 context window 工作，即使使用了 compaction（上下文压缩），也会出现以下问题：

问题	表现
一次做太多	Agent 试图一次性完成整个应用，context 耗尽时功能只做了一半，下一个 session 需要猜测状态
过早宣告完成	看到已有进展后直接声明项目完成，跳过大量功能
环境脏状态	每次 session 结束时留下 bug、未文档化的变更
功能验证不充分	标记功能完成但未做端到端测试

双 Agent 架构

解决方案拆分为两个角色：

1. **Initializer Agent**（首次运行）：创建 `init.sh` 启动脚本、`feature_list.json` 功能需求清单、`claude-progress.txt` 进度文件、初始 git commit
2. **Coding Agent**（后续每次运行）：读取 progress 文件和 git log → 选择一个 feature → 实现 → 测试 → commit → 更新 progress → 循环

关键 insight 是使用 `claude-progress.txt` + git history 让每个新 session 能快速理解当前工作状态。

项目文件结构

以下是完整的实现目录结构，将 Anthropic 的模式映射到 Claude Code 的 Subagent + Skills 体系：

```
your-project/
├── .claude/
│   ├── agents/                # Subagent 定义
│   │   ├── initializer.md       # 初始化 Agent
│   │   ├── coder.md             # 编码 Agent
│   │   ├── tester.md            # 测试 Agent（可选）
```

```
│   │   └── reviewer.md          # 代码审查 Agent（可选）
│   ├── skills/                  # Skill 定义
│   │   ├── init-project/
│   │   │   └── SKILL.md         # 项目初始化 Skill
│   │   ├── progress-tracker/
│   │   │   └── SKILL.md         # 进度管理 Skill
│   │   ├── feature-picker/
│   │   │   └── SKILL.md         # 功能选择 Skill
│   │   └── session-start/
│   │       └── SKILL.md         # Session 启动 Skill
│   └── CLAUDE.md                # 项目级 Claude 指令
├── feature_list.json            # 功能需求清单（JSON）
├── claude-progress.txt          # 进度日志
├── init.sh                      # 开发环境启动脚本
└── ... (项目源码)
```

第一步：创建 Skills

Skills 是 Claude Code 的可复用指令集，存放在 `.claude/skills/<skill-name>/SKILL.md`，Claude 会根据 description 自动判断何时使用。[1]

1. 项目初始化 Skill

这个 Skill 由 Initializer Agent 调用，负责首次环境搭建。

文件：`.claude/skills/init-project/SKILL.md`

```
---
name: init-project
description: Initialize a new long-running agent project with feature list, pro
disable-model-invocation: true
allowed-tools: Read, Write, Edit, Bash, Glob, Grep
---

# Project Initialization for Long-Running Agent

You are setting up the initial environment for a long-running coding project
```

Given a high-level project spec, perform ALL of the following steps:

Step 1: Analyze the spec and generate feature_*list.json*

Create a comprehensive `feature_*list.json` at the project root. Break the
into 50-200+ granular, testable features. Each feature must follow this schem

```json
{
  "features": [
    {
      "id": 1,
      "category": "functional|ui|integration|performance",
      "priority": "high|medium|low",
      "description": "A user can open a new chat and see a welcome message",
      "steps": [
        "Navigate to main interface",
        "Click 'New Chat' button",
        "Verify welcome message appears"
      ],
      "passes": false
    }
  ]
}
```

Rules:

- Every feature starts with `"passes": false`
- Features should be ordered by dependency (foundational first)
- Each feature must be independently testable
- It is UNACCEPTABLE to remove or edit existing features later

Step 2: Create claude-progress.txt

Create `claude-progress.txt` at the project root:

```
# Project Progress Log
## Project: $ARGUMENTS
```

```
## Created: [current date]
## Status: Initialized

### Session Log
- [date] Session 1: Project initialized. Feature list created with N features.
  Environment setup complete. Ready for feature implementation.
```

Step 3: Create init.sh

Write an `init.sh` script that:

- Installs dependencies
- Starts the development server
- Runs a basic smoke test to verify the app loads
- Is idempotent (safe to re-run)

Step 4: Initialize git

```
git init (if not already)
git add -A
git commit -m "chore: initialize project with feature list and progress trackin
```

Step 5: Report summary

Print a summary: total features by category, priority breakdown, and recommended first feature to implement.

```
### 2. 进度追踪 Skill

**文件：`.claude/skills/progress-tracker/SKILL.md`**

```markdown
---
name: progress-tracker
description: Track and report project progress from feature_list.json. Use wh
allowed-tools: Read, Write, Edit, Bash
```
```

Progress Tracker

Manage the project's feature_list.json and claude-progress.txt files.

Commands

When invoked with arguments, handle these modes:

"status" — Show current progress
1. Read `feature_list.json`
2. Calculate: total features, passing, failing, percentage complete
3. Show breakdown by category and priority
4. List the next 5 recommended features to implement

"update [feature-id]" — Mark a feature as passing
1. Read `feature_list.json`
2. Find the feature by ID
3. Set `"passes": true` ONLY if you have verified it works
4. Write back the JSON file
5. NEVER remove or edit feature descriptions — only change the `passes` fiel

"log [message]" — Append to progress file
1. Append a timestamped entry to `claude-progress.txt`
2. Include: what was done, what feature(s) were affected, current pass rate

Rules
- NEVER delete features from feature_list.json
- NEVER edit feature descriptions or steps
- Only mark features as passing after thorough verification
- Always use JSON format (not Markdown) for the feature list

3. Session 启动 Skill

这是每个 Coding Agent session 的入口，对应 Anthropic 文中的 "Getting Up to Speed" 步骤。

文件：`.claude/skills/session-start/SKILL.md`

```
---
name: session-start
description: Start a new coding session by reviewing progress and choosing
disable-model-invocation: true
allowed-tools: Read, Bash, Glob, Grep
---

# Session Startup Procedure

Follow these steps EXACTLY at the start of every session:

## Step 1: Orient yourself
```bash
pwd

# Step 2: Review recent progress

1. Read claude-progress.txt — understand what the last session accomplished
2. Read feature_list.json — see overall progress and remaining work
3. Run git log --oneline -20 — review recent commits

# Step 3: Start the dev environment

1. Read and run init.sh to start the development server
2. Wait for server to be ready
```
```

Step 4: Smoke test

Run a basic end-to-end test to verify the app is in a working state.
If it's broken, FIX IT FIRST before doing anything else.

Step 5: Choose next feature

1. Find the highest-priority feature with `"passes": false`
2. Announce: "I will now implement feature #[id]: [description]"

Do NOT skip any steps. Do NOT start implementing until all 5 steps are done.

```

***

## 第二步：创建 Subagents

Subagent 是 Claude Code 的专门化 AI 助手，每个 Subagent 运行在自己的 conte

### Initializer Agent

**文件：`.claude/agents/initializer.md`**

```markdown
---
name: initializer
description: Initialize a new long-running project. Use ONLY for the first ses
tools: Read, Write, Edit, Bash, Glob, Grep
skills:
  - init-project
  - progress-tracker
model: opus
permissionMode: acceptEdits
memory: project
---

You are the Initializer Agent for a long-running coding project.
```
```

```
Your SOLE job is to set up the project environment so that future Coding
Agents can work effectively across many sessions.

Follow the init-project skill instructions precisely.

When finished, verify:
1. feature_list.json exists and is valid JSON with 50+ features
2. claude-progress.txt exists with the initial entry
3. init.sh exists and is executable
4. An initial git commit has been made
5. The development server starts successfully

Save what you learned about this project to your agent memory.
```

注意 `skills` 字段——它将 Skill 的完整内容注入到 Subagent 的 context 中。Subagent 不会继承父对话的 skills，必须显式声明。[2]

Coding Agent

文件：`.claude/agents/coder.md`

```
---
name: coder
description: Make incremental progress on the project by implementing one
tools: Read, Write, Edit, Bash, Glob, Grep
skills:
  - session-start
  - progress-tracker
model: opus
permissionMode: acceptEdits
memory: project
maxTurns: 100
---

You are the Coding Agent for a long-running project.

## Session Workflow
```

```
1. **Start**: Run the session-start skill to orient yourself
2. **Implement**: Work on ONE feature at a time
3. **Test**: Verify the feature works end-to-end before marking it done
4. **Commit**: Make a descriptive git commit after each feature
5. **Update**: Update feature_list.json and claude-progress.txt
6. **Repeat**: Pick the next feature and continue

## Critical Rules

- Work on ONE feature at a time — never try to implement multiple features
- ALWAYS commit after completing a feature with a descriptive message
- ALWAYS update claude-progress.txt after each feature
- NEVER mark a feature as passing without thorough testing
- NEVER remove or edit features in feature_list.json
- If you find a bug in existing code, FIX IT FIRST
- Leave the codebase in a clean, mergeable state at all times
- When context is running low, commit your work, update progress, and sto

## Memory

Check your agent memory at the start of each session for project-specific
patterns and conventions you've learned previously.
Update your memory with new learnings before ending a session.
```

Tester Agent（可选的专门化 Agent）

文件：`.claude/agents/tester.md`

```
---
name: tester
description: Verify features work end-to-end. Use after implementing a featu
tools: Read, Bash, Glob, Grep
skills:
  - progress-tracker
model: sonnet
permissionMode: plan
```

```
memory: project
mcpServers:
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
---

You are the Testing Agent. Your job is to verify that implemented features
work correctly through end-to-end testing.

When invoked:
1. Read the feature description and test steps from feature_list.json
2. Start the dev server using init.sh
3. Execute each test step using browser automation (Playwright)
4. Report PASS or FAIL with detailed evidence
5. For failures, describe what went wrong and what you observed

You are READ-ONLY — do not modify any code. Only report results.
```

这个 Tester Agent 示范了如何通过 `mcpServers` 给 Subagent 配置专门的 MCP 服务器（如 Playwright 用于浏览器自动化）。`permissionMode: plan` 使其只能读不能写。[2]

第三步：配置 CLAUDE.md

文件：`.claude/CLAUDE.md`

```
# Long-Running Agent Project

## Agent Architecture

This project uses a two-agent pattern for long-running development:

1. **Initializer Agent** (`/agents initializer`): Run ONCE at project start
2. **Coding Agent** (`/agents coder`): Run for every subsequent session
3. **Tester Agent** (`/agents tester`): Optional, for end-to-end verification
```

```
## Key Files

- `feature_list.json` — Source of truth for all features (JSON, never Markdow
- `claude-progress.txt` — Human-readable progress log
- `init.sh` — Idempotent dev environment setup script

## Rules

- NEVER delete or modify feature descriptions in feature_list.json
- ALWAYS commit after completing a feature
- ALWAYS update progress files after each session
- Work on ONE feature at a time
- Fix bugs before implementing new features
- Leave the codebase clean and mergeable at session end
```

第四步：运行流程

首次运行（Initializer）

在 Claude Code 交互模式中：

```
Use the initializer agent to set up this project: Build a clone of
claude.ai with chat interface, conversation history, theme switching,
and streaming responses.
```

或使用 headless 模式（-p 标志）进行无人值守运行：[3]

```
claude -p "Use the initializer agent to set up this project based on spec.md" \
  --allowedTools "Read" "Write" "Edit" "Bash(npm *)" "Bash(git *)" \
  --max-turns 50
```

后续运行（Coding Agent 循环）

每次 session：

```
Use the coder agent to make progress on this project
```

Headless 循环运行：

```bash
#!/bin/bash
# run-agent-loop.sh

MAX_SESSIONS=20

for i in $(seq 1 $MAX_SESSIONS); do
  echo "=== Session $i of $MAX_SESSIONS ==="

  claude -p "Use the coder agent to implement the next feature. \
    Work on exactly one feature, test it, commit, and update progress." \
    --allowedTools "Read" "Write" "Edit" "Bash(npm *)" "Bash(node *)" \
    "Bash(git *)" "Bash(npx *)" \
    --max-turns 80 \
    --max-budget-usd 2.00

  # Check if all features pass
  REMAINING=$(cat feature_list.json | python3 -c "
import json, sys
data = json.load(sys.stdin)
remaining = sum(1 for f in data['features'] if not f['passes'])
print(remaining)
  ")

  if [ "$REMAINING" -eq 0 ]; then
    echo "All features complete!"
    break
  fi

  echo "$REMAINING features remaining. Starting next session..."
```

```
  sleep 5
done
```

`--max-turns` 限制每个 session 的操作次数，`--max-budget-usd` 设置花费上限，这是无人值守运行的安全保障。[4]

第五步：进阶模式

使用 Agent Teams 实现并行开发

对于更大型的项目，Claude Code 的 **Agent Teams** 功能可以让多个 Claude 实例并行工作。与 Subagent 不同，Agent Teams 中的 Teammate 拥有完全独立的 context window，可以互相通信和协调。[5]

```
Create an agent team for this project:
- A lead agent that reads the feature list and assigns work
- 3 coding teammates, each working on independent features
- Each teammate should follow the coder agent workflow

Only assign features that have no dependencies on each other.
```

Agent Teams 使用共享任务列表（task list）进行协调，每个 Teammate 完成任务后自动通知 Lead。[5]

使用 Persistent Memory 跨 Session 学习

Subagent 的 `memory` 字段可以启用持久化记忆：[2]

Scope	路径	适用场景
user	~/.claude/agent-memory/<agent-name>/	跨所有项目记忆
project	.claude/agent-memory/<agent-name>/	项目特定知识，可版本控制
local	.claude/agent-memory-local/<agent-name>/	项目特定但不入版本控制

启用后，Subagent 会自动获得读写 memory 目录的能力，并在 context 中包含 MEMORY.md 的前 200 行。建议在 Subagent prompt 中加入指示让它主动维护自己的知识库。[2]

Skill 的 context: fork 模式

对于需要隔离运行的 Skill（如研究性任务），使用 context: fork 让 Skill 在独立 Subagent 中运行：[1]

```
---
name: deep-research
description: Research a topic thoroughly
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:
1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Summarize findings with specific file references
```

此模式下 Skill 内容成为 Subagent 的 task prompt，结果被总结后返回主对话。[1]

使用 Hooks 进行生命周期管理

Subagent 支持在 frontmatter 中定义 hooks，在工具使用前后执行自定义逻辑：[2]

```yaml
---
name: safe-coder
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-command.sh"
  PostToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "./scripts/run-linter.sh"
---
```

也可以在项目级 `settings.json` 中配置 `SubagentStart` / `SubagentStop` 事件来响应 Subagent 的启停。[2]

故障排查

问题	原因	解决方案
Skill 不触发	`description` 关键词不匹配	检查描述是否包含自然语言关键词；用 `/skill-name` 直接调用[1]
Subagent 无法使用 Skill	未在 frontmatter 声明 skills	Subagent 不继承父对话的 skills，必须在 skills 字段显式列出[2]
Skills 数量过多被截断	描述超出 context budget	Claude Code 按 context window 2% 分配预算；设置 `SLASH_COMMAND_TOOL_CHAR_BUDGET` 环境变量调整[1]
Headless 模式权限问题	--allowedTools 未覆盖所需工具	精确列出所需工具而非用 --dangerously-skip-permissions[6][4]
Agent 跨 session 状态丢失	未使用 `memory` 字段	为 Subagent 配置 `memory: project` 启用持久化记忆[2]

架构对比

特性	Subagent	Agent Teams	Skills
Context	独立 context window[2]	完全独立的 Claude 实例[5]	在主对话或 fork context 中运行[1]
通信	只能向主 Agent 返回结果[5]	Teammate 之间可直接通信[5]	返回结果到调用者
最佳场景	专注任务，只需返回结果[2]	需要讨论和协作的复杂工作[5]	可复用的工作流和领域知识[1]
Token 成本	较低：结果被总结后返回[5]	较高：每个 Teammate 是独立实例[5]	最低：在现有 context 中运行
持久化	支持 `memory` 字段[2]	继承 CLAUDE.md 和 Skills[5]	不直接持久化，依赖外部文件

完整工作流总结

将 Anthropic 的长时运行 Agent 模式映射到 Claude Code 技术栈的对应关系如下：[1][2]

Anthropic 原始概念	Claude Code 实现
Initializer Agent 的首次运行 prompt	`.claude/agents/initializer.md` Subagent + `init-project` Skill
Coding Agent 的每次运行 prompt	`.claude/agents/coder.md` Subagent + `session-start` Skill
`feature_list.json` 管理	`progress-tracker` Skill 封装增删查改逻辑
claude-progress.txt 维护	Subagent 的 memory: project + `progress-tracker` Skill
`init.sh` 开发环境管理	`init-project` Skill 生成，`session-start` Skill 每次调用
多 context window 循环	claude -p headless 模式 + shell 脚本循环[3]
端到端测试验证	`tester` Subagent + Playwright MCP Server[2]
多 Agent 协作（未来方向）	Agent Teams 并行开发[5]

References

1. Extend Claude with skills - Claude Code Docs - Add context: fork to your frontmatter when you want a skill to run in isolation. The skill content b...
2. Create custom subagents - Claude Code Docs - Create and use specialized AI subagents in Claude Code for task-specific workflows and improved cont...
3. Run Claude Code programmatically - Claude Code Docs - The CLI was previously called “headless mode.” The -p flag and all CLI options work the same way. To...
4. Claude Code Can Work While You Sleep | wmedia.es - With the -p flag, Claude Code becomes a headless agent you can schedule with cron. Combined with --a...

5. Orchestrate teams of Claude Code sessions - Agent teams let you coordinate multiple Claude Code instances working together. One session acts as ...
6. Claude Code as an autonomous agent: the permission model ... - The flag that enables headless mode: -p claude -p "your instruction ... I built 18 autonomous agents...